L01, Group 06
Melody Yang 400307007
Cynthia Sun 400238765

McMaster University

MECHTRON 3TB4

Lab 1 Report

Date: February 4, 2026

Lab 1 Report

Overview

In Lab 1, the main objective was to become familiarized with the basic digital logic design and Verilog HDL by implementing, simulating, and compiling the code and synthesizing it in Quartus. It was then implemented onto the DE1-SoC FPGA board. This lab focused on understanding Verilog's use of combinational and sequential logic and how the design is mapped onto a physical FPGA. This lab was divided into 3 main parts to further one's understanding of the Verilog language, the software to hardware integration of the seven-segment display, and lastly, the integration of these components into a complete system.

In part 1, several basic logic modules were designed and tested including D flip-flops, various control signals (both synchronous reset and enable), a D-latch, a 4-bit multiplexer, and a 4-bit counter. These modules introduced concepts such as clock logic, active-low control signals, edge-triggered behaviours, and enable control.

In part 2, a seven-segment decoder was created to convert 4-bit binary inputs into the corresponding seven-segment display outputs. The decoder was used to display numbers 0-9 as well as the first five letters of each of the students' first names. Due to the limited display combinations of a seven-segment decoder, some liberties were taken in the creation of a few of the English alphabets.

Finally, in part 3, modules from part 1 and 2 were integrated together. The switches and keys were used as inputs, the multiplexers chose what to display, the counters generated changing values, and all the results were displayed on the DE1-SoC board's HEX seven-segment display.

Purpose of the DE1-SoC.qsf File

The DE1-SoC.qsf file contains Quartus project settings for the DE1-SoC FPGA board used in the lab, which include device configuration, source files, and pin assignments. These are used to map the design signals to the physical hardware components available on the DE1-SoC board, such as switches, buttons, LEDs, and clocks, making it easier to efficiently connect design variables to the available hardware resources during the labs.

Part 1: Basic Logic Blocks

One-bit data width D flip-flop

Table 1. Truth table for the one-bit data width D flip-flop.

clk	d (input)	q(n+1)
0	0	q(n)
0	1	q(n)
1	0	0
1	1	1

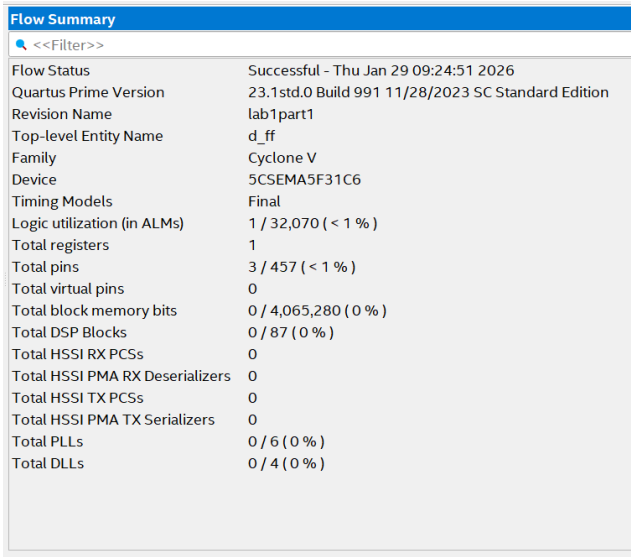


Figure 1. Compilation report of the one-bit data width D flip-flop module.

Table 2. Summary for Components for the one-bit data width D flip-flop in Lab 1 Part 1.

Feature	Quantity
Total number of logic elements	1
Total number of registers	1
Total number of pins	3
Max number of logic elements that can fit on FPGA	32,070

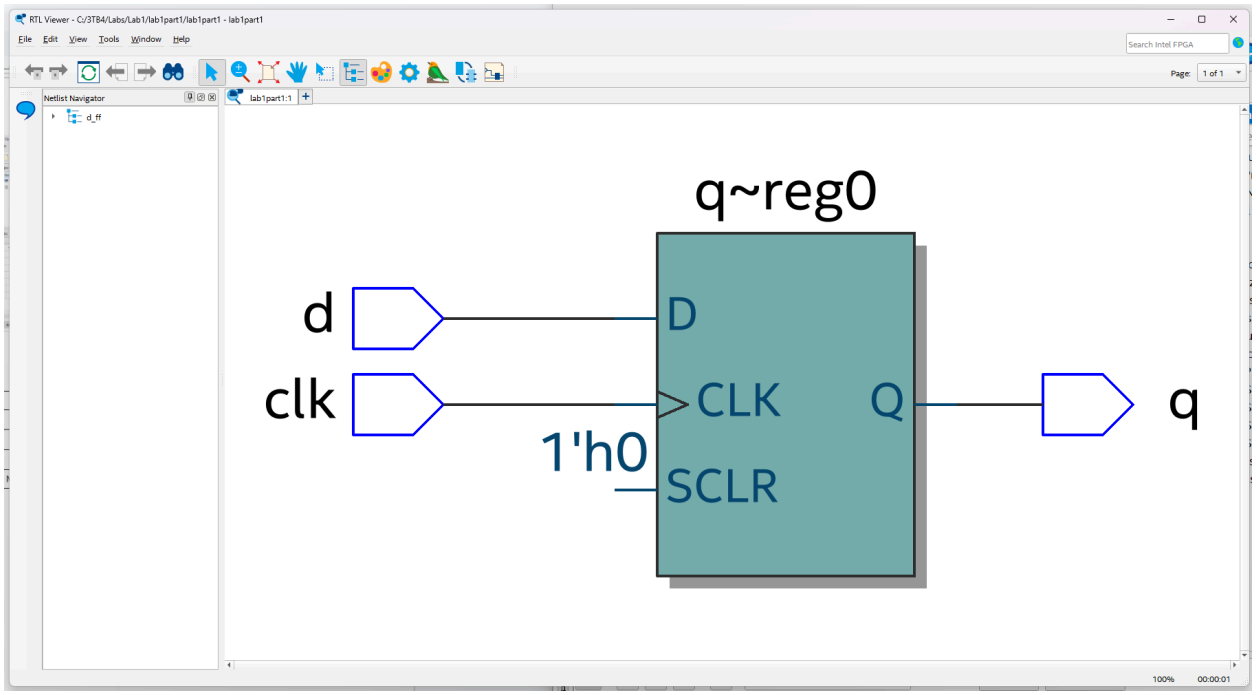


Figure 2. Screenshot of the implemented one-bit data width D flip-flop module.

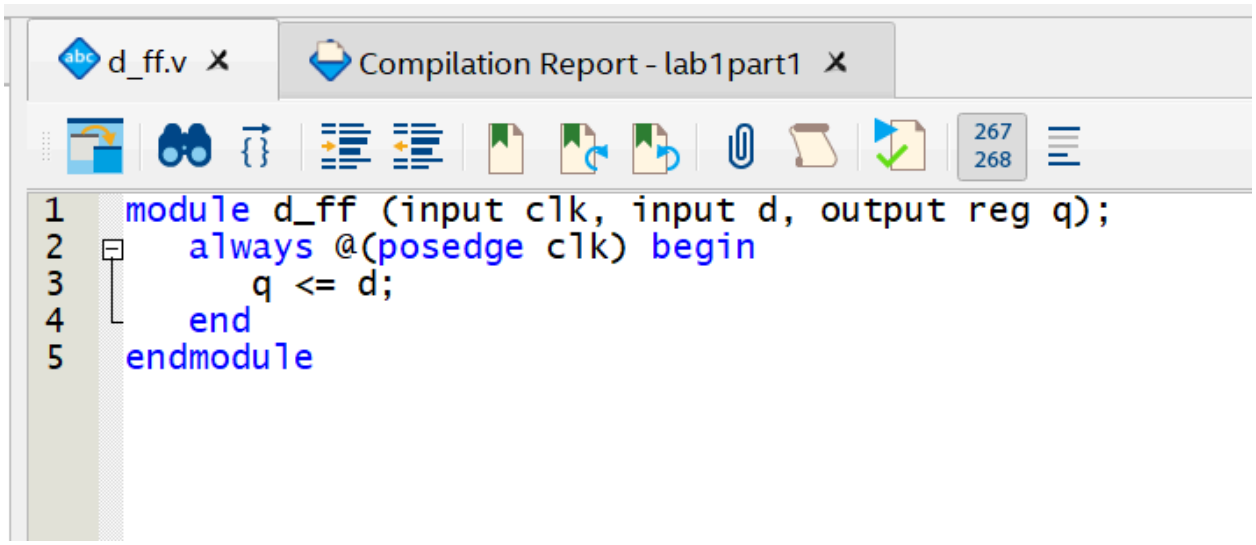


Figure 3. Screenshot of the module for a one-bit data width D flip-flop module.

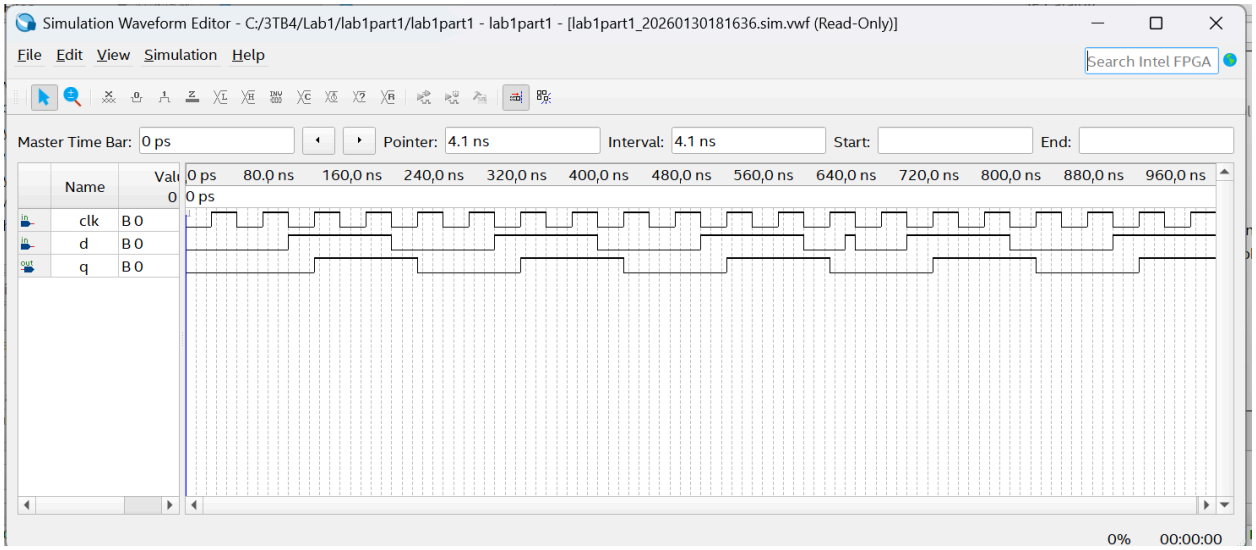


Figure 4. Simulation results for the one-bit data width D flip-flop module.

One-bit data width D flip-flop with active low synchronous reset

Table 3. Truth table for the one-bit data width D flip-flop with active low synchronous reset.

clk	reset	d (input)	q(n+1)
0	X	X	q(n)
1	0	X	0
1	1	0	0
1	1	1	1

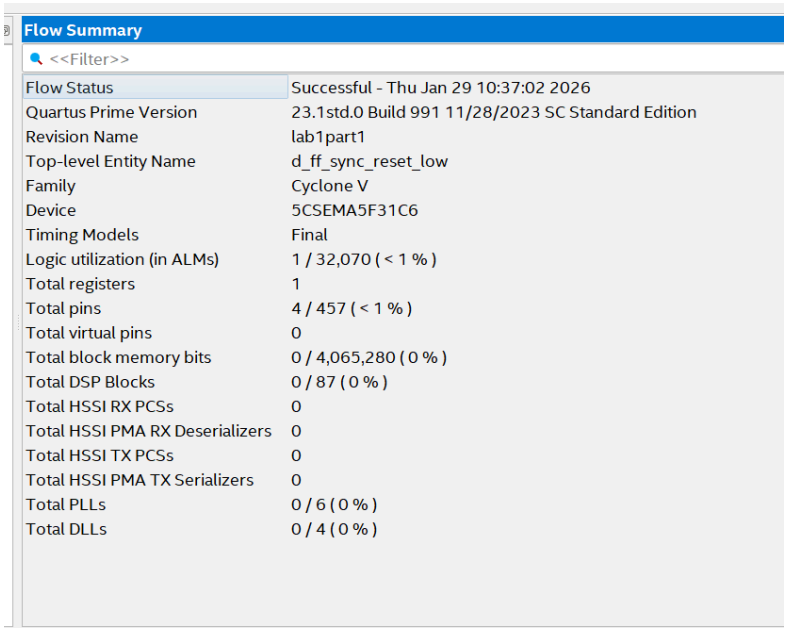


Figure 5. Compilation report of the one-bit data width D flip-flop with active low synchronous reset.

Table 4. Summary for Components for the one-bit data width D flip-flop with active low synchronous reset in Lab 1 Part 1.

Feature	Quantity
Total number of logic elements	1
Total number of registers	1
Total number of pins	4
Max number of logic elements that can fit on FPGA	32,070

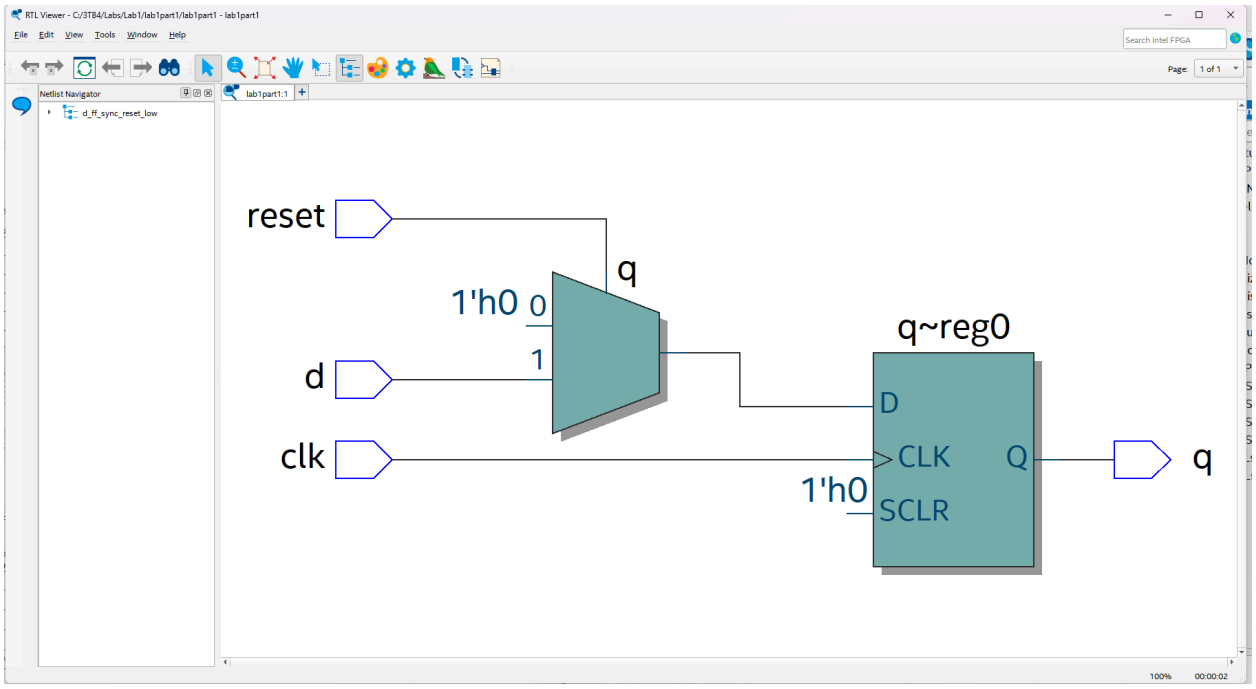


Figure 6. Screenshot of the implemented one-bit data width D flip-flop with active low synchronous reset.

d_ff_sync_reset_low.v

Compilation Report - lab1part1

```
1  module d_ff_sync_reset_low (input clk, input d, input reset, output reg q);
2     always @(posedge clk) begin
3        if (!reset)
4           q <= 1'b0;
5        else
6           q <= d;
7     end
8  endmodule
```

Figure 7. Screenshot of the module for a one-bit data width D flip-flop with active low synchronous reset.

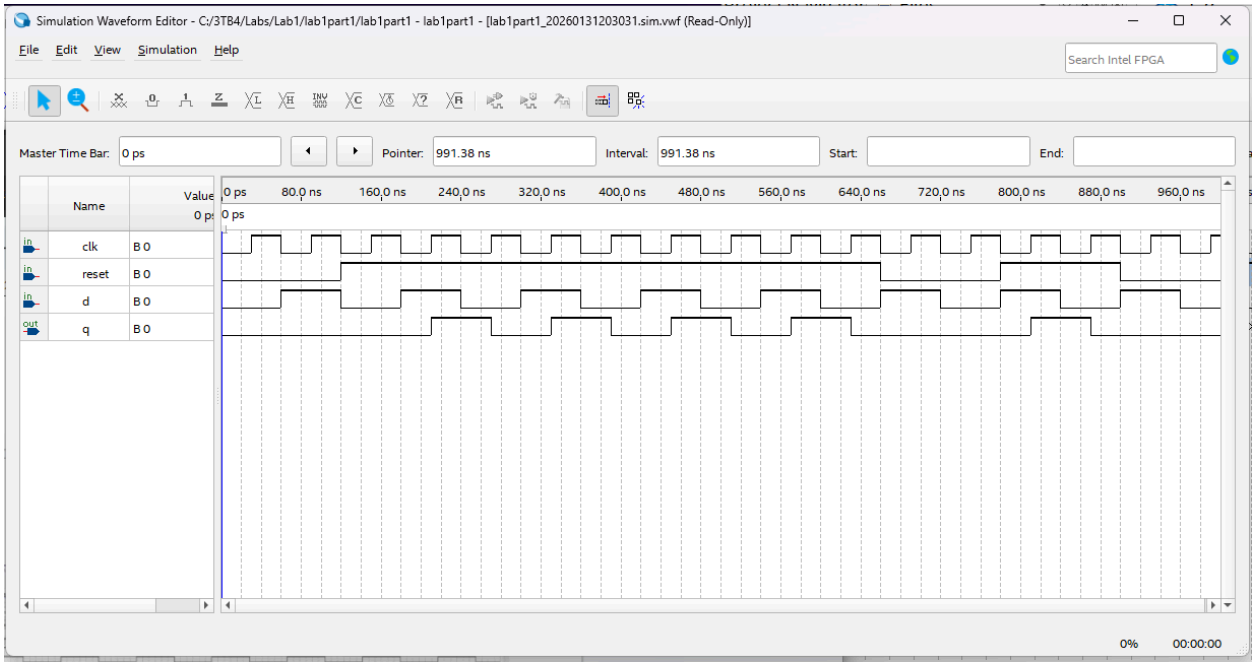


Figure 8. Simulation results for the one-bit data width D flip-flop with active low synchronous reset.

One-bit data width D flip-flop with active low synchronous reset and active low enable

Table 5. Truth table for the one-bit data width D flip-flop with active low synchronous reset and active low enable.

clk	reset	enable	d (input)	q(n+1)
0	X	X	X	q(n)
1	0	X	X	0
1	1	0	0	0
1	1	0	1	1
1	1	1	X	q(n)

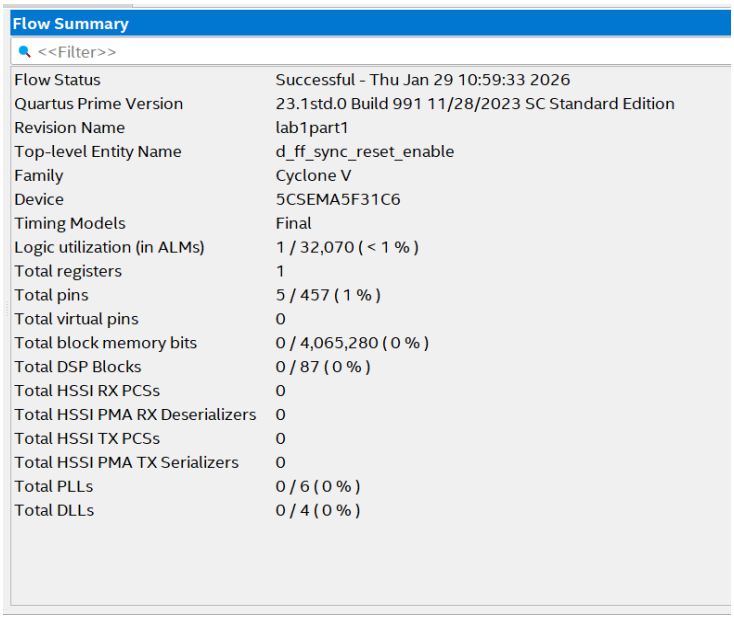


Figure 9. Compilation report of the one-bit data width D flip-flop with active low synchronous reset and active low enable.

Table 6. Summary for components for the one-bit data width D flip-flop with active low synchronous reset and active low enable in Lab 1 Part 1.

Feature	Quantity
Total number of logic elements	1
Total number of registers	1
Total number of pins	5
Max number of logic elements that can fit on FPGA	32,070

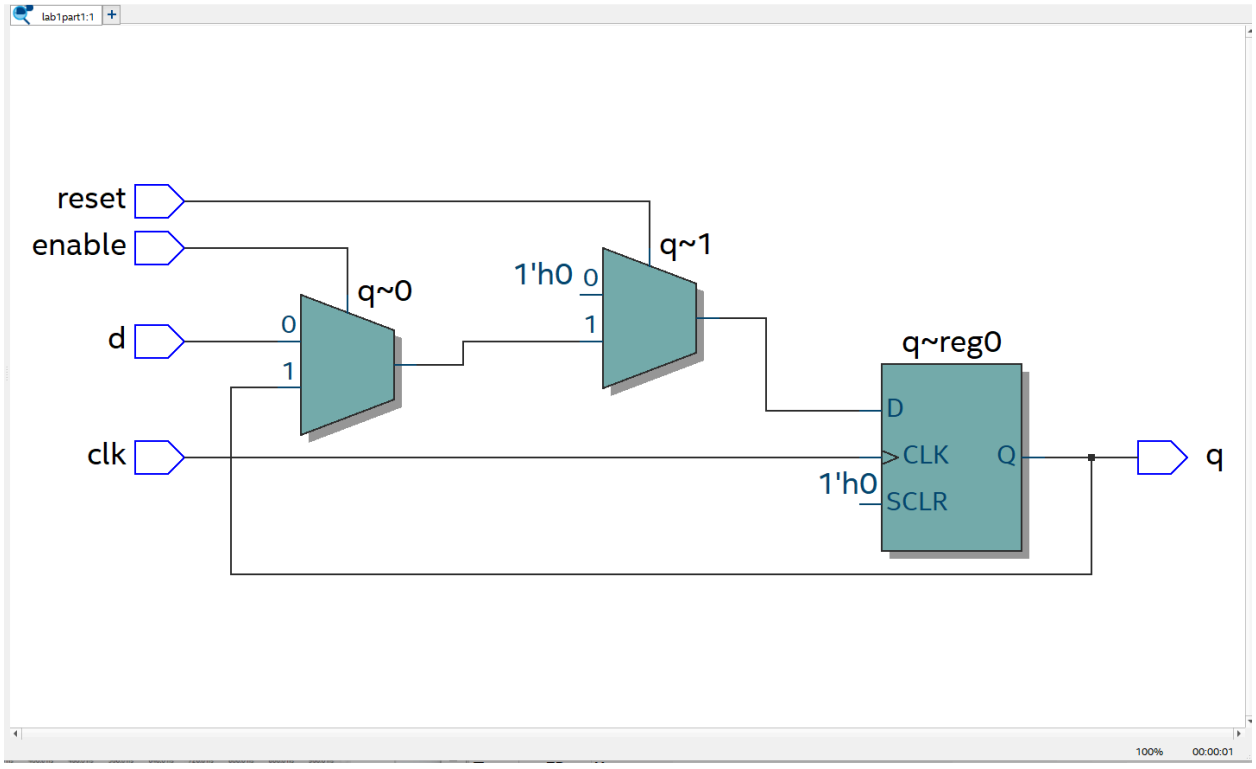


Figure 10. Screenshot of the implemented one-bit data width D flip-flop with active low synchronous reset and active low enable.

Compilation Report - lab1part1 | d_ff_sync_reset_enable.v

```
1  module d_ff_sync_reset_enable(input clk, input d, input reset, input enable, output reg q);
2     always@(posedge clk) begin
3        if (!reset)
4           q <= 1'b0;
5        else if (!enable)
6           q <= d;
7     end
8  endmodule
9
```

Figure 11. Screenshot of the module for one-bit data width D flip-flop with active low synchronous reset and active low enable.

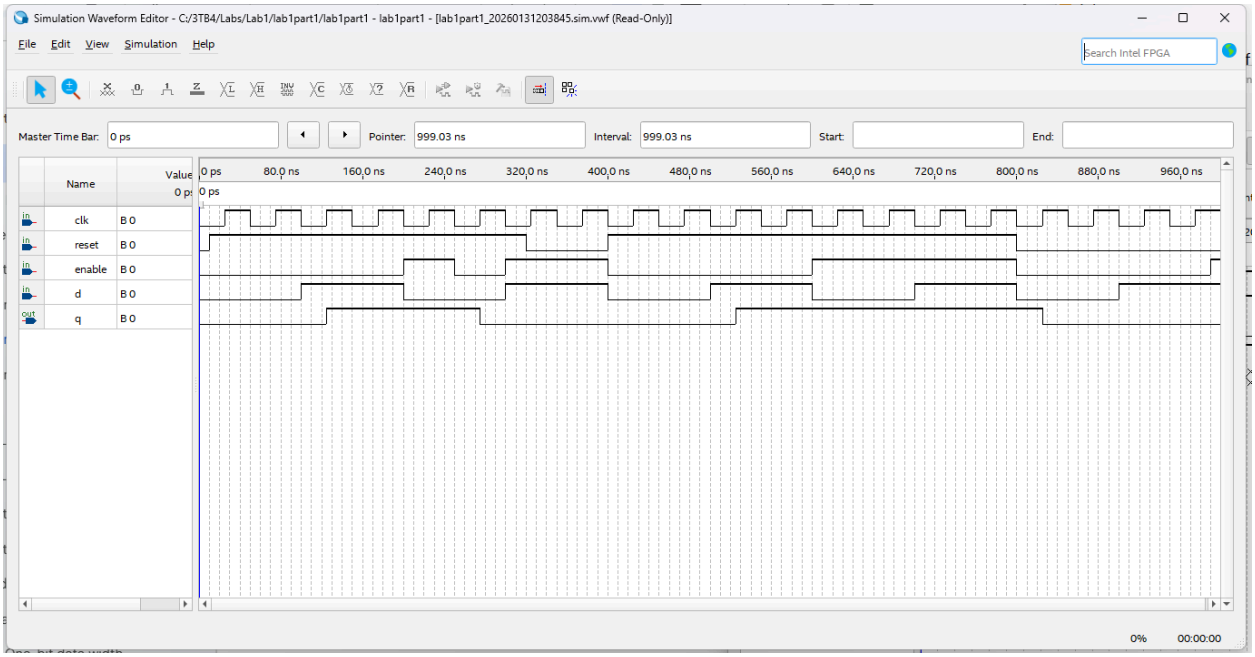


Figure 12. Simulation results for the one-bit data width D flip-flop with active low synchronous reset and active low enable.

D latch with synchronous enable control

Table 7. Truth table for the D latch with synchronous enable control.

clk	enable	d (input)	q(n+1)
0	X	X	q(n)
1	1	0	0
1	1	1	1
1	0	X	q(n)

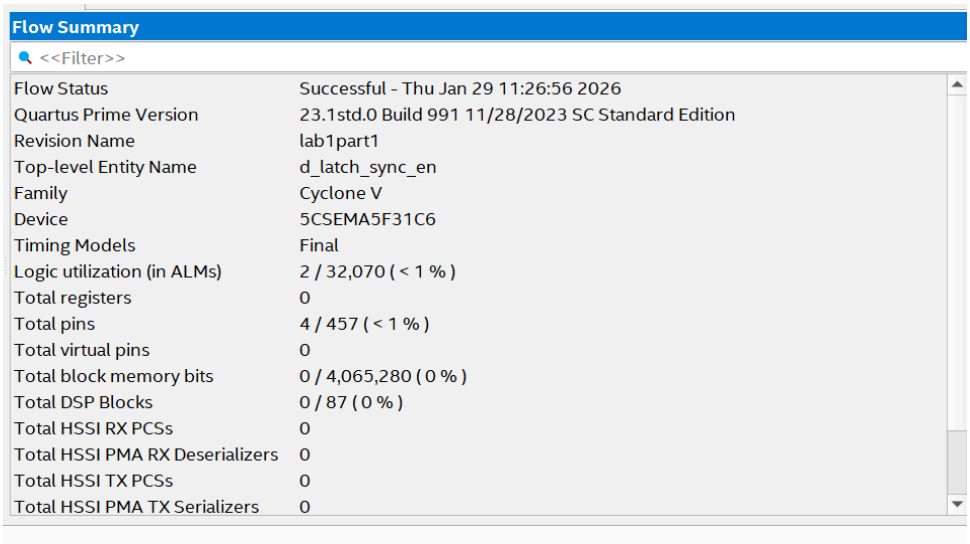


Figure 13. Compilation report of the D latch with synchronous enable control.

Table 8. Summary for components for the D latch with synchronous enable control in Lab 1 Part 1.

Feature	Quantity
Total number of logic elements	2
Total number of registers	0
Total number of pins	4
Max number of logic elements that can fit on FPGA	32,070

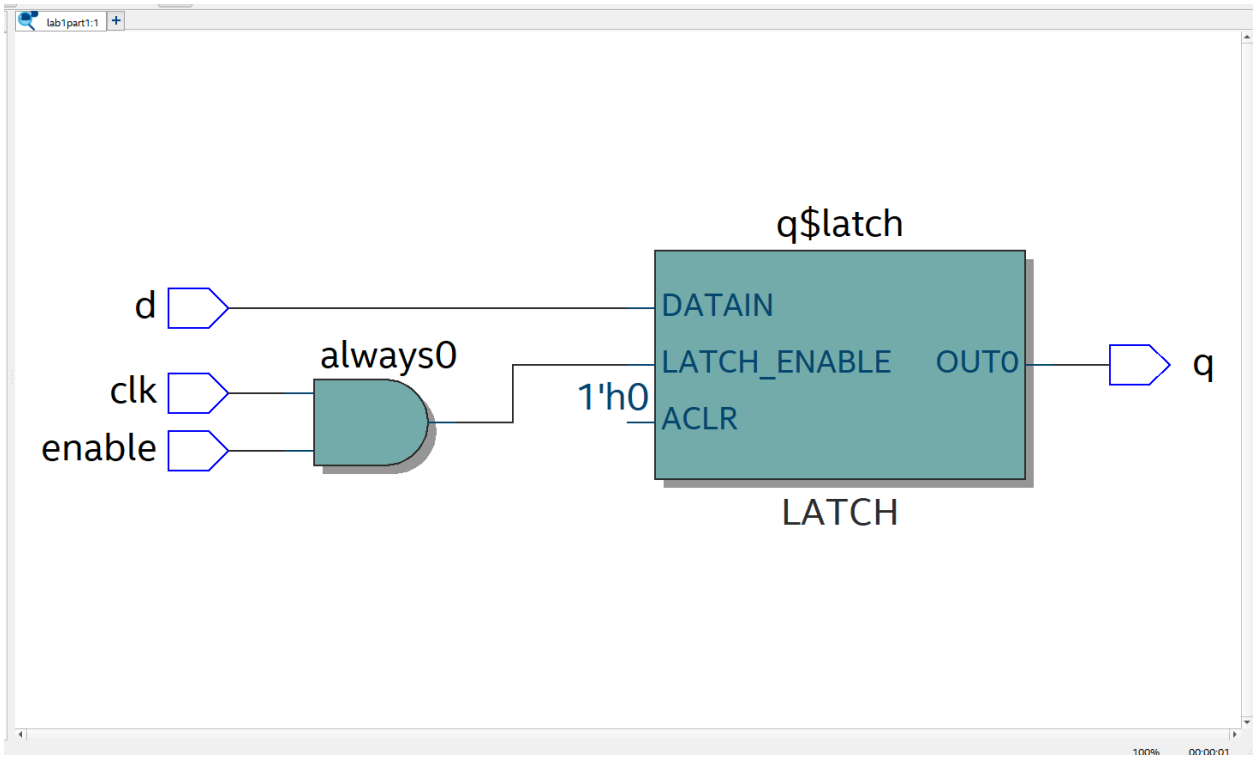


Figure 14. Screenshot of the implemented D latch with synchronous enable control.

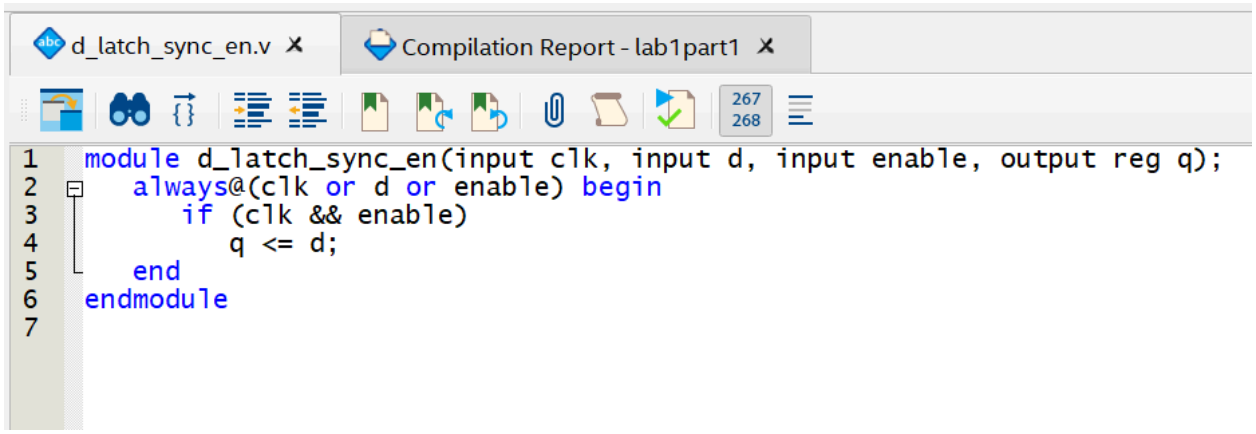


Figure 15. Screenshot of the module for D latch with synchronous enable control.

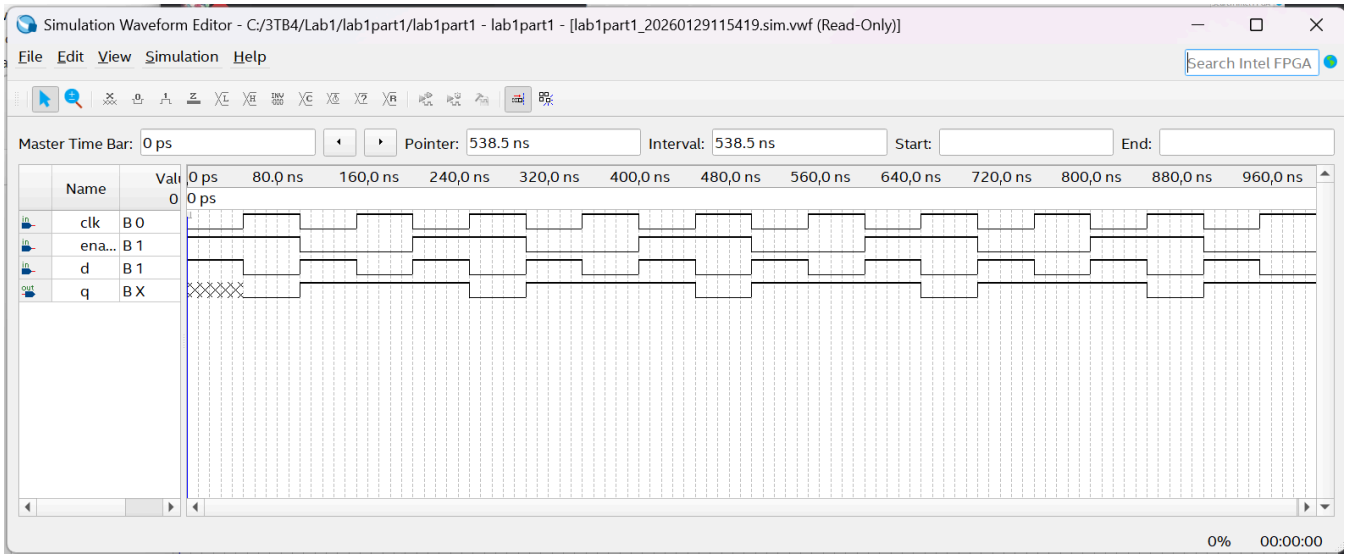


Figure 16. Simulation results for the D latch with synchronous enable control.

4-to-1 multiplexer

Table 9. Truth table for the 4-to-1 multiplexer.

select	in[3]	in[2]	in[1]	in[0]	output
00	X	X	X	0	0
00	X	X	X	1	1
01	X	X	0	X	0
01	X	X	1	X	1
10	X	0	X	X	0
10	X	1	X	X	1
11	0	X	X	X	0
11	1	X	X	X	1

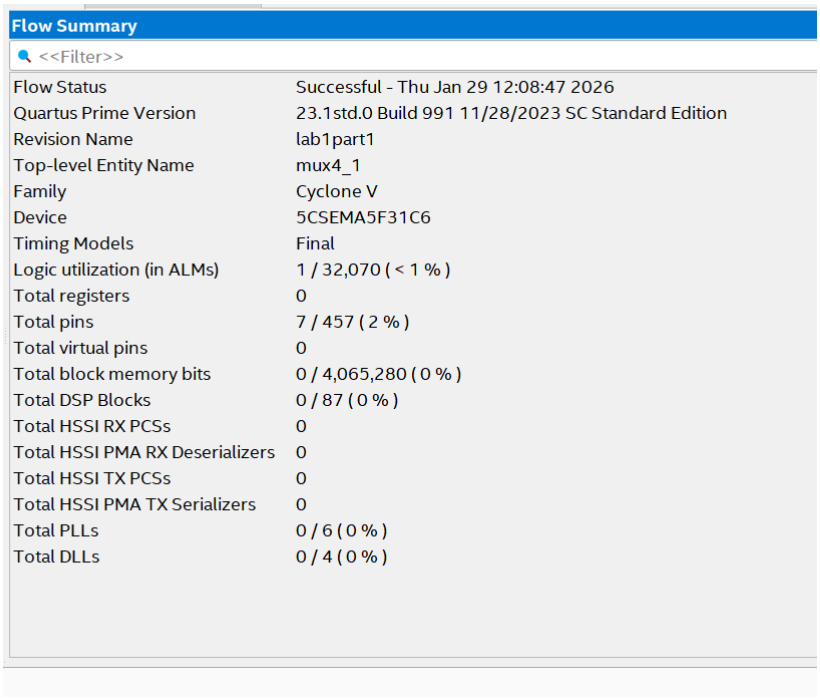


Figure 17. Compilation report of the 4-to-1 multiplexer.

Table 10. Summary for components for the 4-to-1 multiplexer in Lab 1 Part 1.

Feature	Quantity
Total number of logic elements	1
Total number of registers	0
Total number of pins	7
Max number of logic elements that can fit on FPGA	32,070

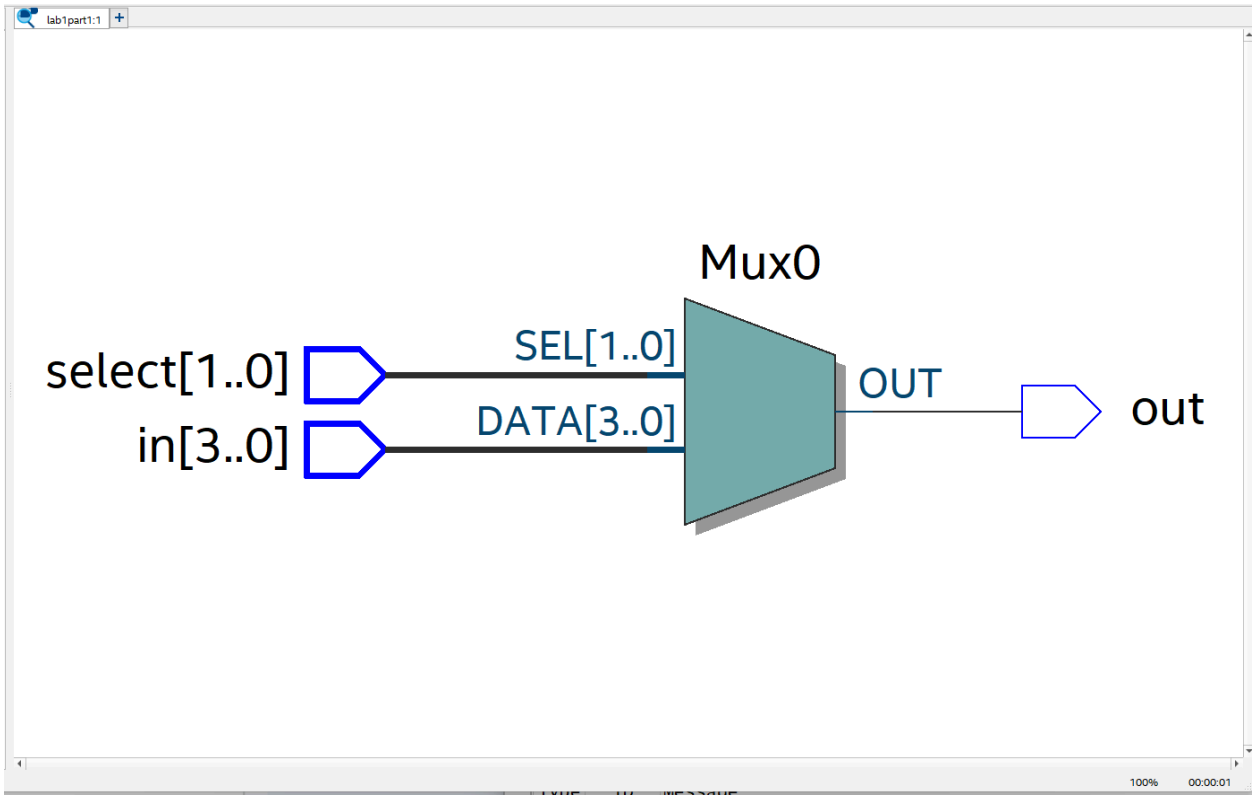


Figure 18. Screenshot of the implemented 4-to-1 multiplexer.

Compilation Report - lab1part1 | mux4_1.v

```
1  module mux4_1(input [1:0] select, input [3:0] in, output reg out);
2     always@(*) begin
3        case (select)
4           2'b00: out = in[0];
5           2'b01: out = in[1];
6           2'b10: out = in[2];
7           2'b11: out = in[3];
8           default: out = 1'b0;
9        endcase
10    end
11 endmodule
12
```

Figure 19. Screenshot of the module for 4-to-1 multiplexer.

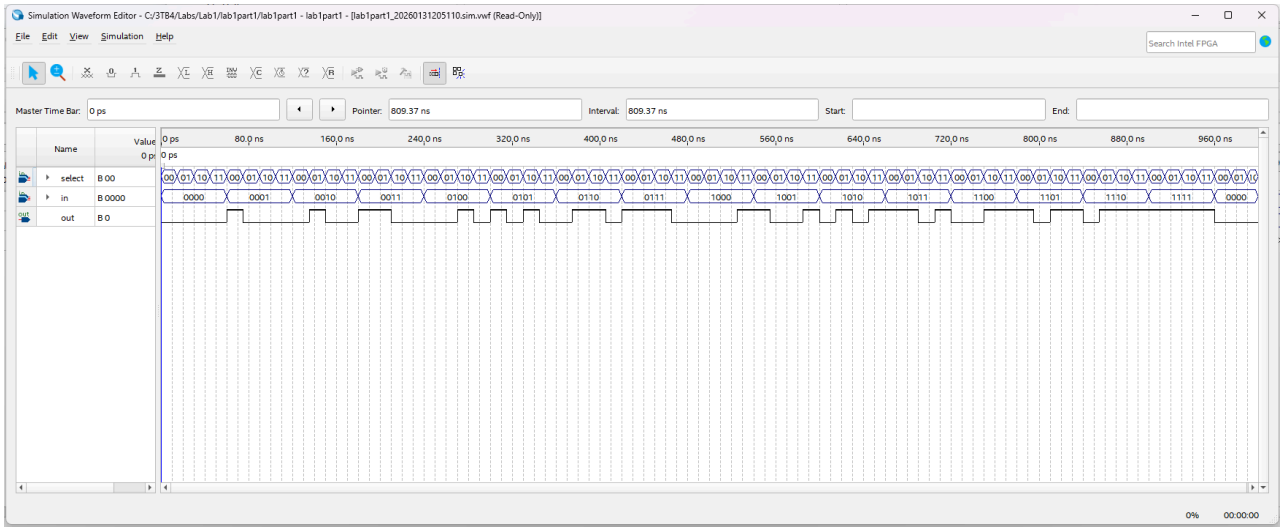


Figure 20. Simulation results for the 4-to-1 multiplexer.

4-bit counter with reset and enable controls

Table 11. Truth table for the 4-bit counter with reset and enable controls.

clk	reset	enable	output q(n+1)
0	X	X	q(n)
1	0	0	q(n)
1	0	1	q(n) + 1
1	1	X	0000

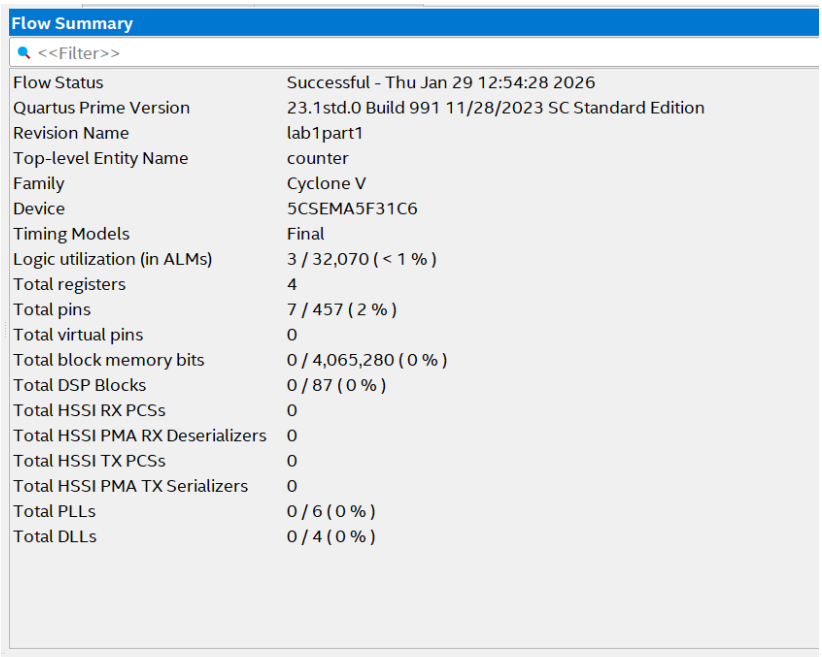


Figure 21. Compilation report of the 4-bit counter with reset and enable controls.

Table 12. Summary for components for the 4-bit counter with reset and enable controls in Lab 1 Part 1.

Feature	Quantity
Total number of logic elements	3
Total number of registers	4
Total number of pins	7
Max number of logic elements that can fit on FPGA	32,070

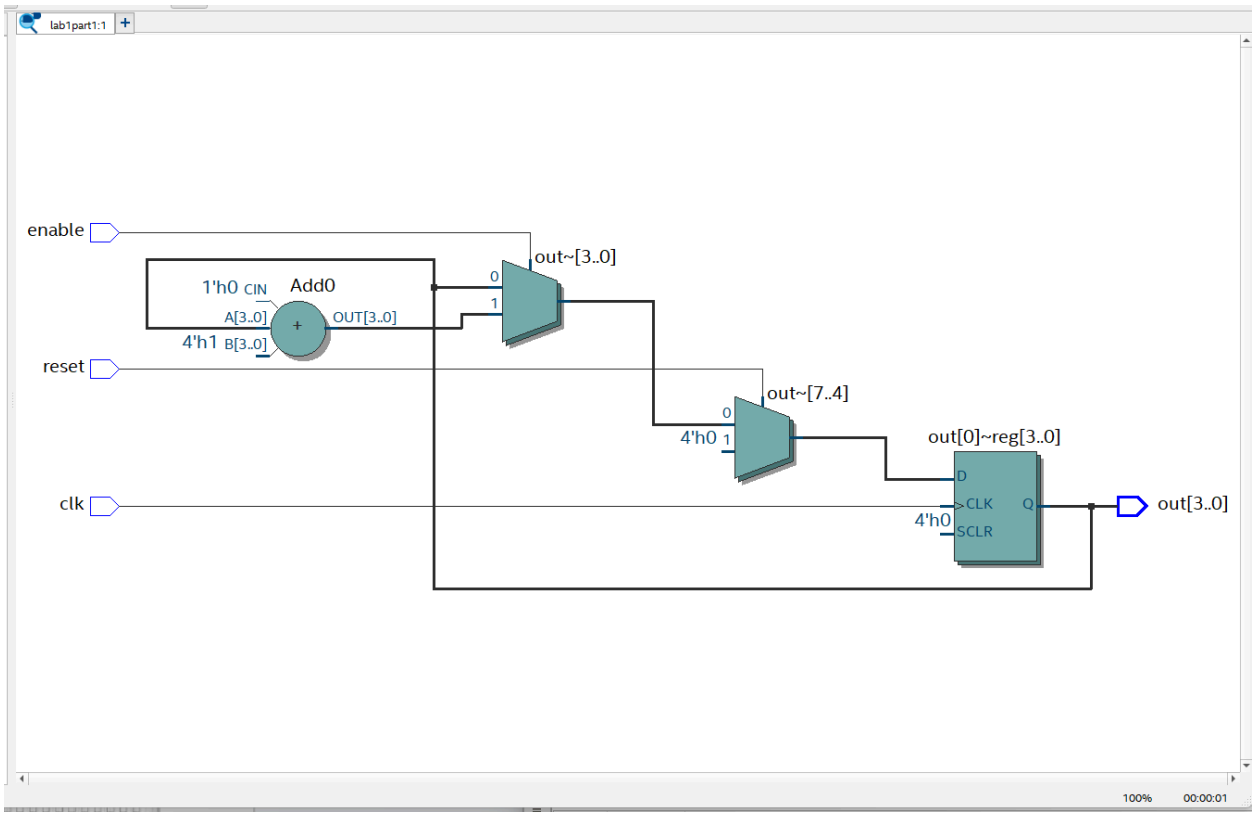


Figure 22. Screenshot of the implemented 4-bit counter with reset and enable controls.

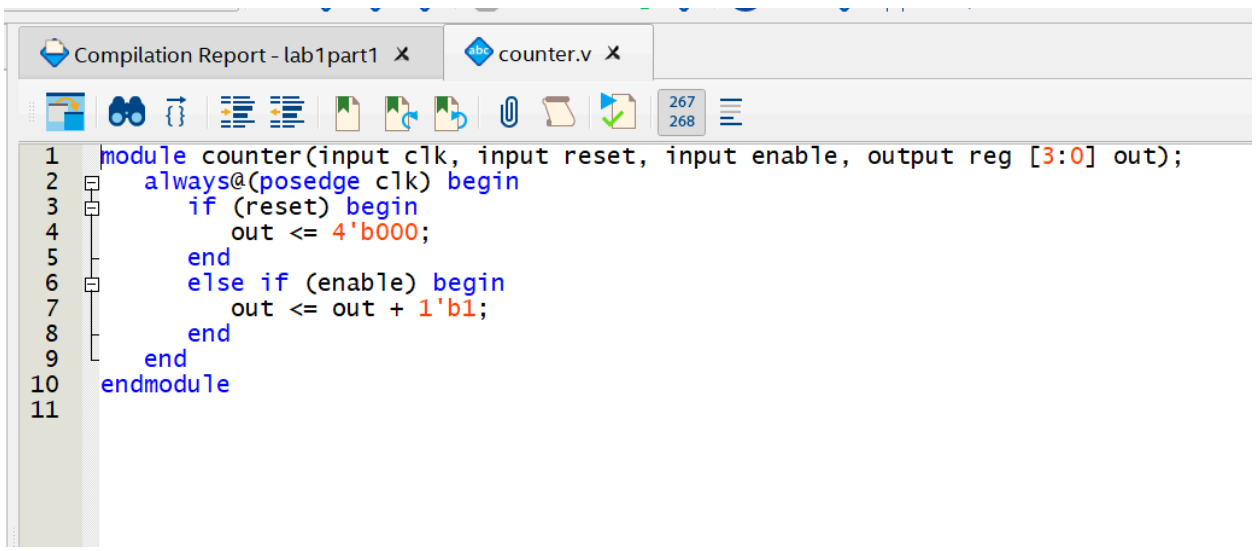


Figure 23. Screenshot of the module for the 4-bit counter with reset and enable controls.

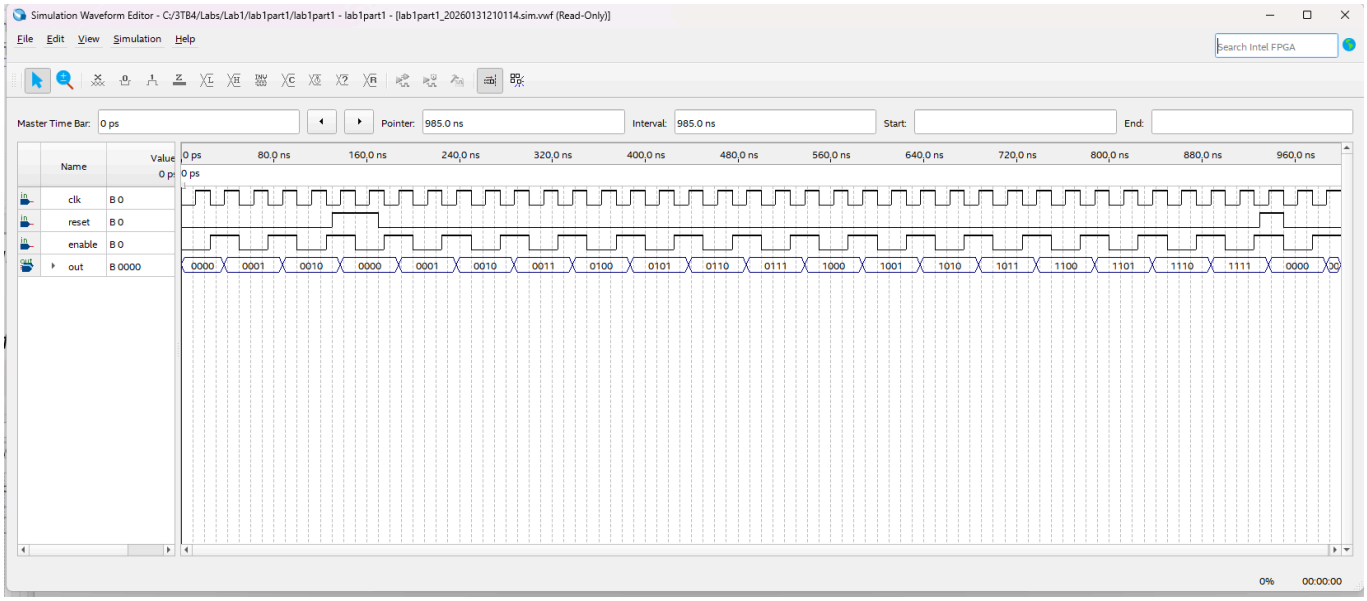


Figure 24. Simulation results for the 4-bit counter with reset and enable controls.

Part 2: Seven-Segment Decoder

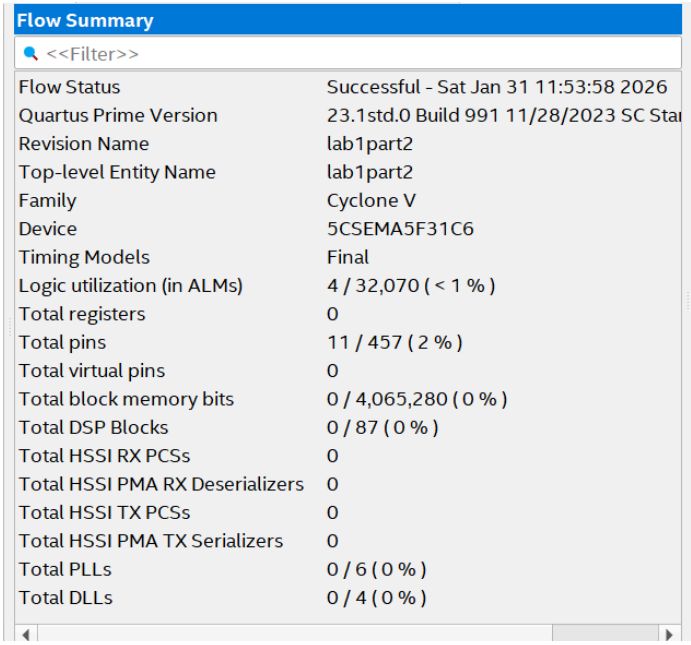


Figure 25. Compilation report for Lab 1 Part 2.

Table 13. Summary for Components for Lab 1 Part 2.

Feature	Quantity
Total number of logic elements	4
Total number of registers	0
Total number of pins	11
Max number of logic elements that can fit on FPGA	32,070

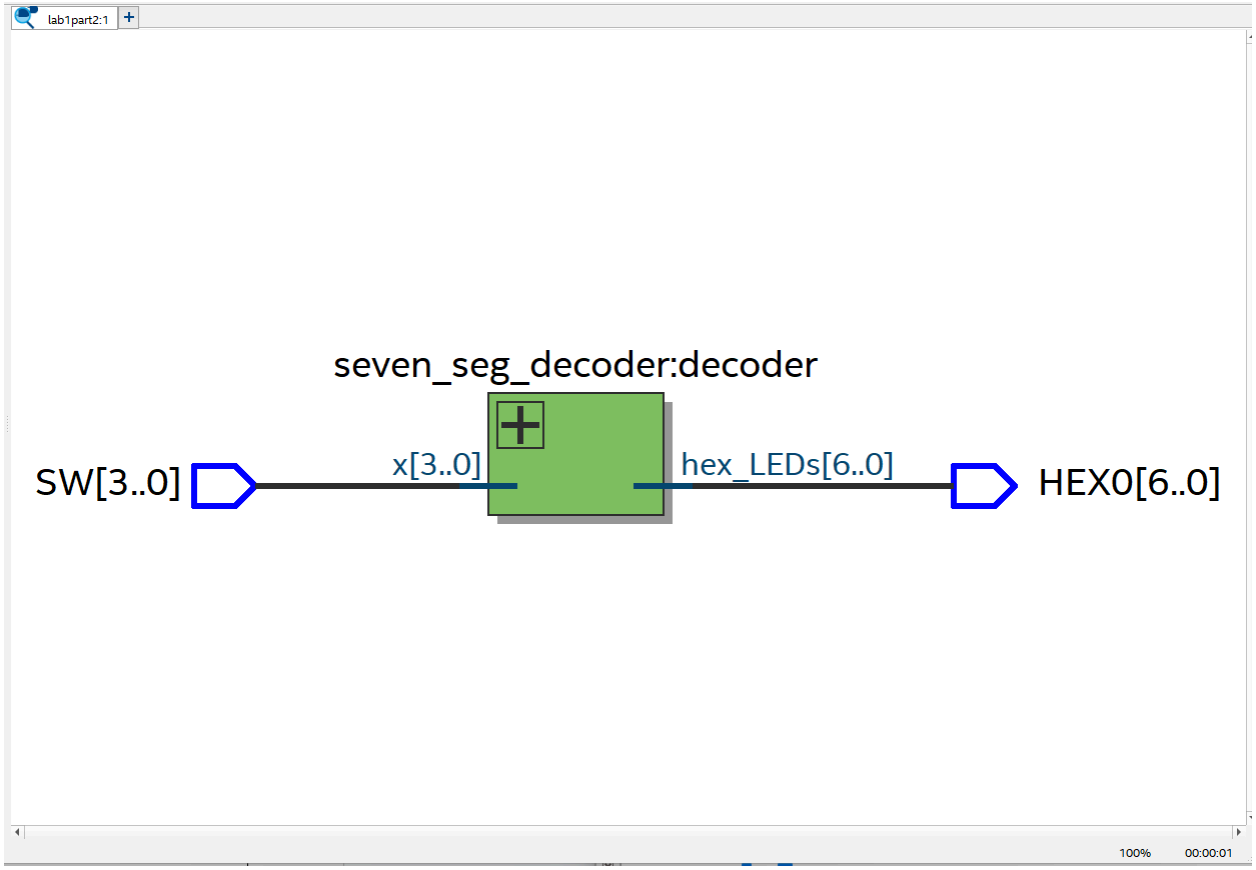


Figure 26. Screenshot of the implemented seven-segment decoder.

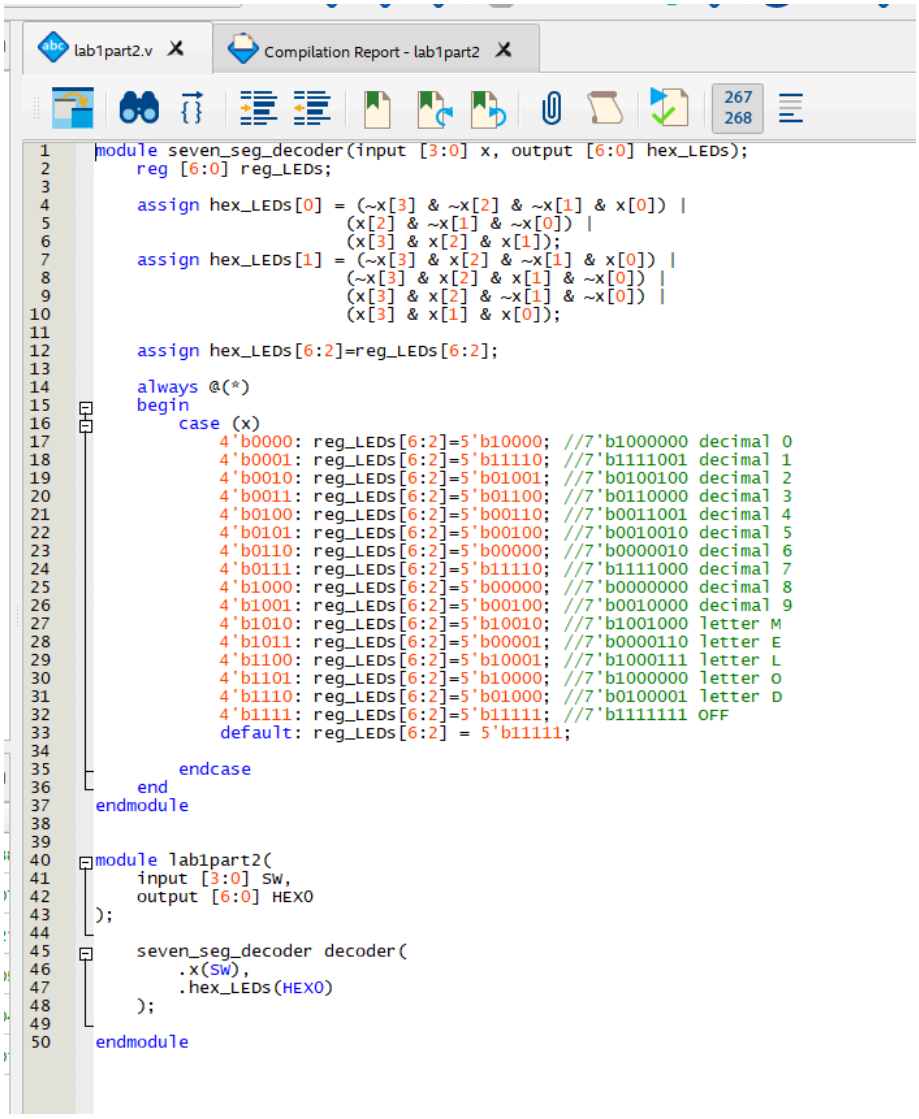
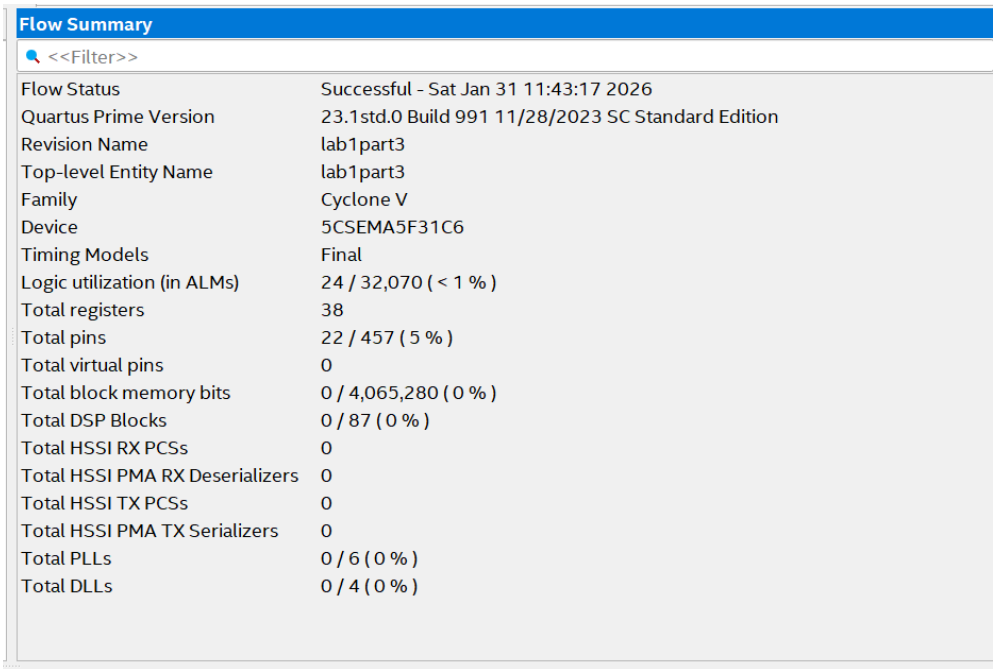


Figure 27. Screenshot of the module for the seven-segment decoder for MELOD.

Part 3: Circuit



Flow Summary

<<Filter>>

Flow Status	Successful - Sat Jan 31 11:43:17 2026
Quartus Prime Version	23.1std.0 Build 991 11/28/2023 SC Standard Edition
Revision Name	lab1part3
Top-level Entity Name	lab1part3
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	24 / 32,070 (< 1 %)
Total registers	38
Total pins	22 / 457 (5 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)
Total HSSI RX PCSs	0
Total HSSI PMA RX Deserializers	0
Total HSSI TX PCSs	0
Total HSSI PMA TX Serializers	0
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 28. Compilation report for Lab 1 Part 3.

Table 14. Summary for Components for Lab 1 Part 3.

Feature	Quantity
Total number of logic elements	24
Total number of registers	38
Total number of pins	22
Max number of logic elements that can fit on FPGA	32,070

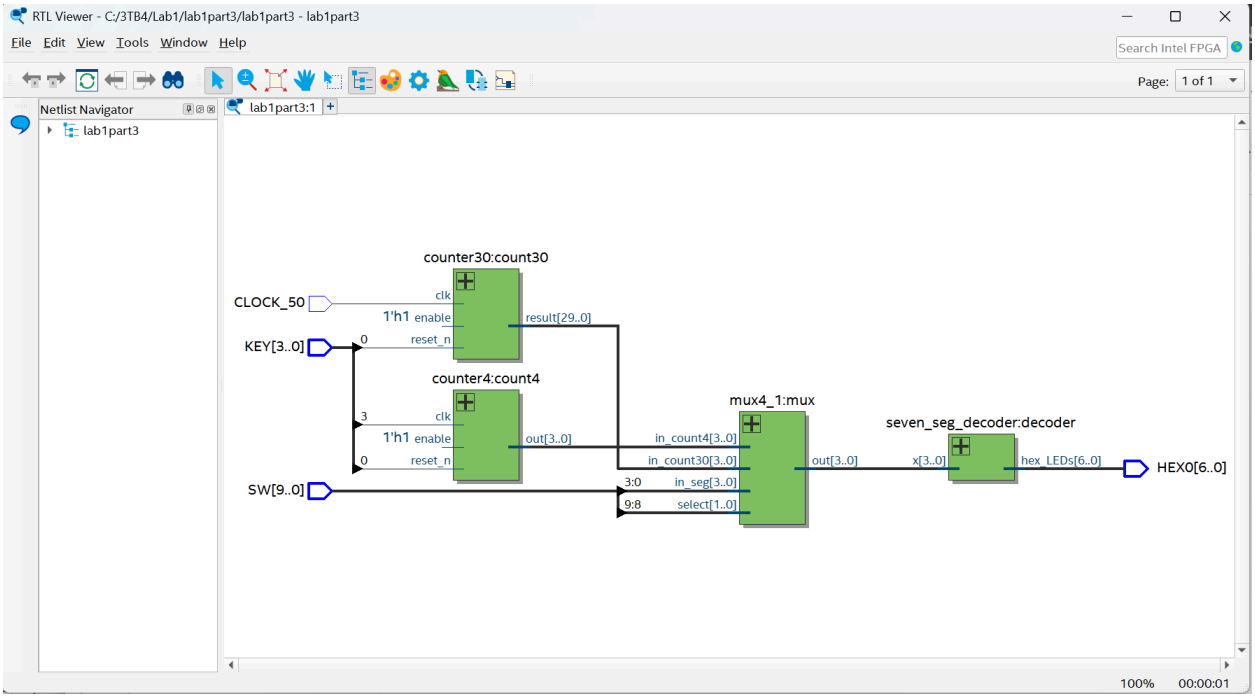


Figure 28. RTL view of the circuit.

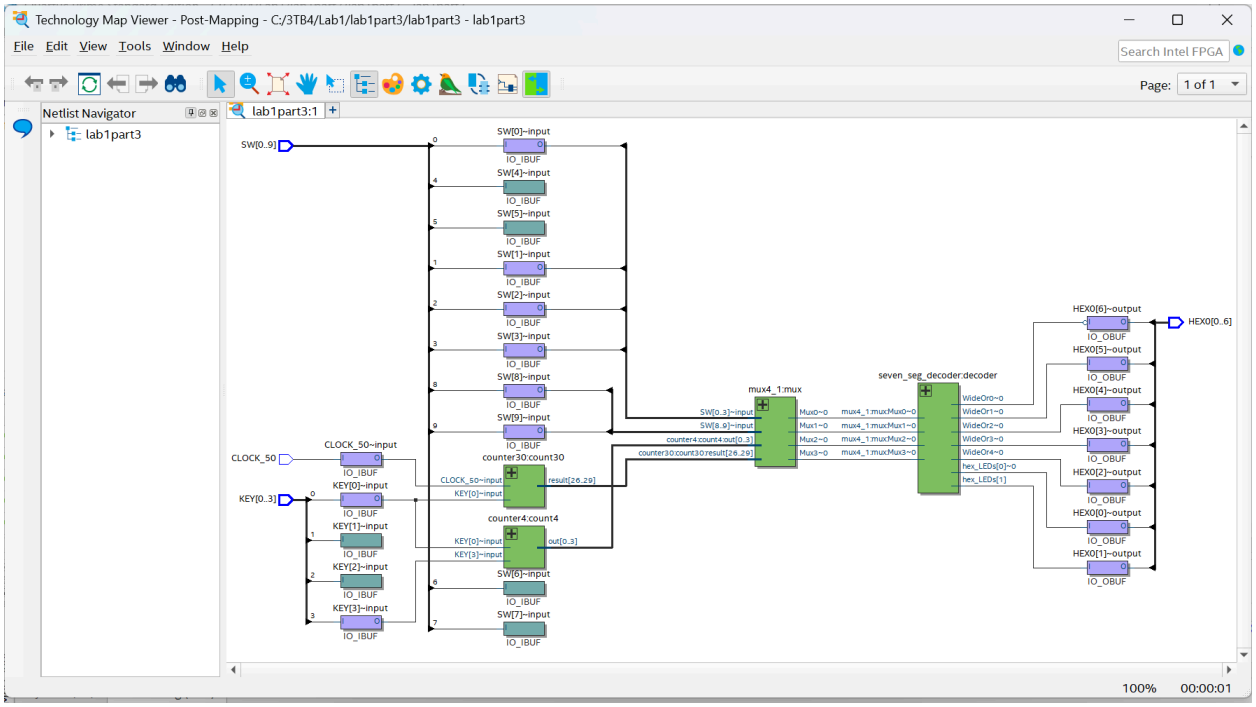


Figure 29. Technology map viewer (post-mapping) of the circuit.

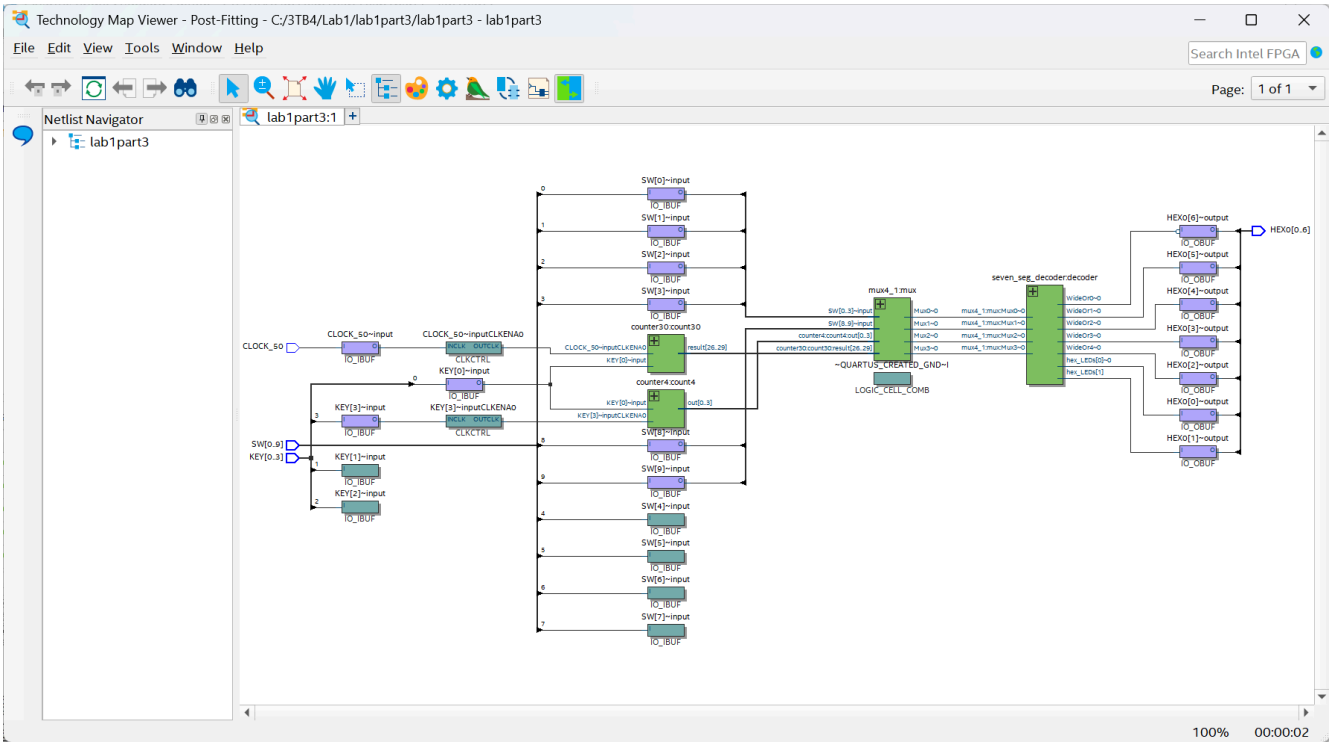


Figure 30. Technology map viewer (post-fitting) of the circuit.

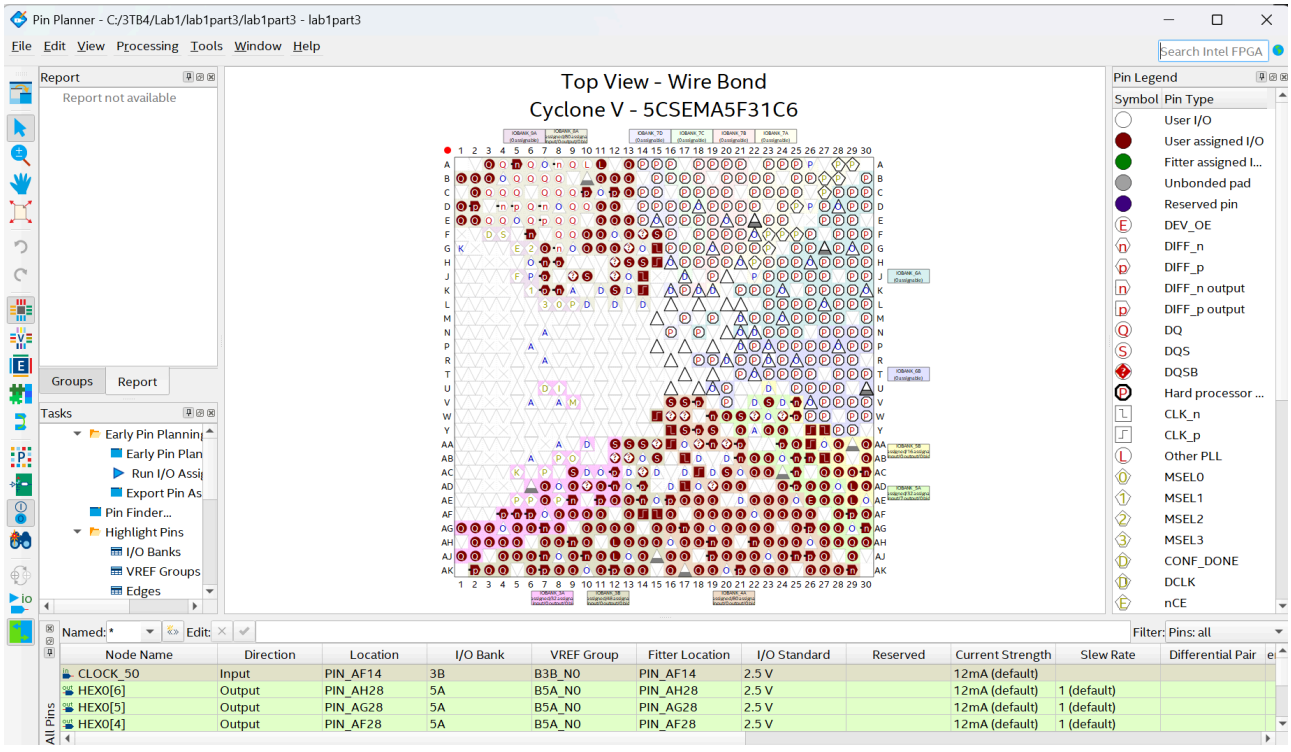
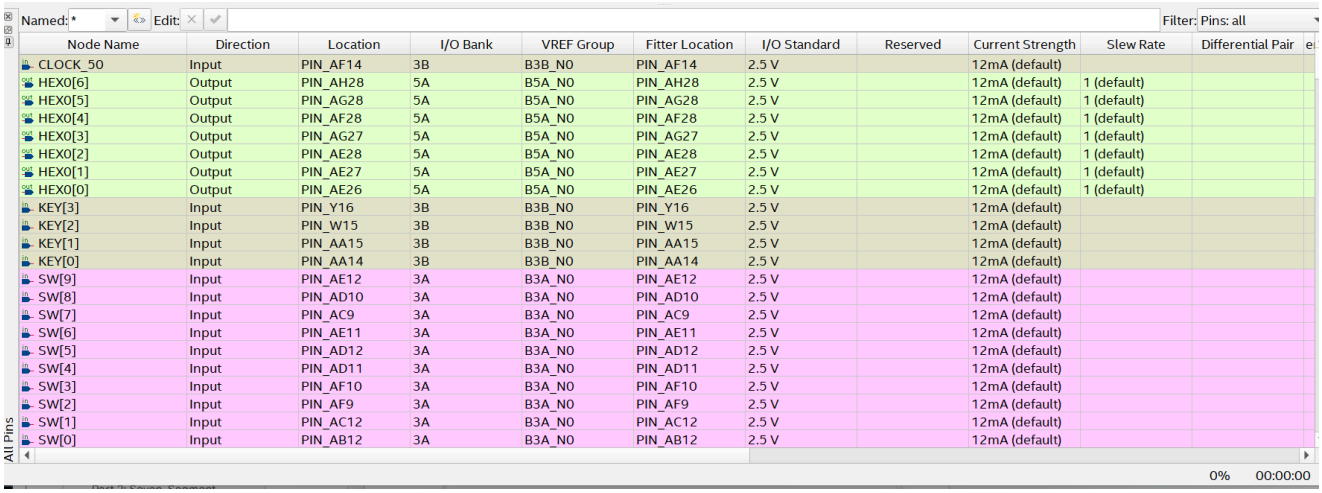


Figure 31a. Pin planner image.



Named: * Edit: Filter: Pins: all

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
CLOCK_50	Input	PIN_AF14	3B	B3B_N0	PIN_AF14	2.5 V		12mA (default)		
HEX0[6]	Output	PIN_AH28	5A	B5A_N0	PIN_AH28	2.5 V		12mA (default)	1 (default)	
HEX0[5]	Output	PIN_AG28	5A	B5A_N0	PIN_AG28	2.5 V		12mA (default)	1 (default)	
HEX0[4]	Output	PIN_AF28	5A	B5A_N0	PIN_AF28	2.5 V		12mA (default)	1 (default)	
HEX0[3]	Output	PIN_AG27	5A	B5A_N0	PIN_AG27	2.5 V		12mA (default)	1 (default)	
HEX0[2]	Output	PIN_AE28	5A	B5A_N0	PIN_AE28	2.5 V		12mA (default)	1 (default)	
HEX0[1]	Output	PIN_AE27	5A	B5A_N0	PIN_AE27	2.5 V		12mA (default)	1 (default)	
HEX0[0]	Output	PIN_AE26	5A	B5A_N0	PIN_AE26	2.5 V		12mA (default)	1 (default)	
KEY[3]	Input	PIN_Y16	3B	B3B_N0	PIN_Y16	2.5 V		12mA (default)		
KEY[2]	Input	PIN_W15	3B	B3B_N0	PIN_W15	2.5 V		12mA (default)		
KEY[1]	Input	PIN_AA15	3B	B3B_N0	PIN_AA15	2.5 V		12mA (default)		
KEY[0]	Input	PIN_AA14	3B	B3B_N0	PIN_AA14	2.5 V		12mA (default)		
SW[9]	Input	PIN_AE12	3A	B3A_N0	PIN_AE12	2.5 V		12mA (default)		
SW[8]	Input	PIN_AD10	3A	B3A_N0	PIN_AD10	2.5 V		12mA (default)		
SW[7]	Input	PIN_AC9	3A	B3A_N0	PIN_AC9	2.5 V		12mA (default)		
SW[6]	Input	PIN_AE11	3A	B3A_N0	PIN_AE11	2.5 V		12mA (default)		
SW[5]	Input	PIN_AD12	3A	B3A_N0	PIN_AD12	2.5 V		12mA (default)		
SW[4]	Input	PIN_AD11	3A	B3A_N0	PIN_AD11	2.5 V		12mA (default)		
SW[3]	Input	PIN_AF10	3A	B3A_N0	PIN_AF10	2.5 V		12mA (default)		
SW[2]	Input	PIN_AF9	3A	B3A_N0	PIN_AF9	2.5 V		12mA (default)		
SW[1]	Input	PIN_AC12	3A	B3A_N0	PIN_AC12	2.5 V		12mA (default)		
SW[0]	Input	PIN_AB12	3A	B3A_N0	PIN_AB12	2.5 V		12mA (default)		

All Pins

0% 00:00:00

Figure 31b. Pin planner table.

Code

Part 1: Basic Logic Blocks

One-bit data width D flip-flop

```
module d_flipflop(input clk, input d, output reg q);
	always@(posedge clk) begin
		q <= d;
	end
endmodule
```

One-bit data width D flip-flop with active low synchronous reset

```
module d_ff_sync_reset_low(input clk, input d, input reset, output reg q);
	always@(posedge clk) begin
		if (!reset)
			q <= 1'b0;
		else
			q <= d;
	end
endmodule
```

One-bit data width D flip-flop with active low synchronous reset and active low enable

```
module d_ff_sync_reset_enable(input clk, input d, input reset, input enable, output reg q);
	always@(posedge clk) begin
		if (!reset)
			q <= 1'b0;
		else if (!enable)
			q <= d;
	end
```

```
endmodule
```

D latch with synchronous enable control

```
module d_latch_sync_en(input clk, input d, input enable, output reg q);
    always@(*) begin
        if (clk && enable)
            q <= d;
    end
endmodule
```

4-to-1 multiplexer

```
module mux4_1(input [1:0] select, input [3:0] in, output reg out);
    always@(*) begin
        case (select)
            2'b00: out = in[0];
            2'b01: out = in[1];
            2'b10: out = in[2];
            2'b11: out = in[3];
            default: out = 1'b0;
        endcase
    end
endmodule
```

4-bit counter with reset and enable controls

```
module counter(input clk, input reset, input enable, output reg [3:0] out);
    always@(posedge clk) begin
        if (reset) begin
            out <= 4'b0000;
        end
        else if (enable) begin
            out <= out + 1'b1;
```

```
            end
        end
endmodule
```

Part 2: Seven-Segment Decoder

```
module seven_seg_decoder(input [3:0] x, output [6:0] hex_LEDs);
    reg [6:0] reg_LEDs;

    assign hex_LEDs[0] = (~x[3] & ~x[2] & ~x[1] & x[0]) |
                         (x[2] & ~x[1] & ~x[0]) |
                         (x[3] & x[2] & x[1]);
    assign hex_LEDs[1] = (~x[3] & x[2] & ~x[1] & x[0]) |
                         (~x[3] & x[2] & x[1] & ~x[0]) |
                         (x[3] & x[2] & ~x[1] & ~x[0]) |
                         (x[3] & x[1] & x[0]);

    assign hex_LEDs[6:2]=reg_LEDs[6:2];

    always @(*)
    begin
        case (x)
            4'b0000: reg_LEDs[6:2]=5'b10000; //7'b1000000 decimal 0
            4'b0001: reg_LEDs[6:2]=5'b11110; //7'b1111001 decimal 1
            4'b0010: reg_LEDs[6:2]=5'b01001; //7'b0100100 decimal 2
            4'b0011: reg_LEDs[6:2]=5'b01100; //7'b0110000 decimal 3
            4'b0100: reg_LEDs[6:2]=5'b00110; //7'b0011001 decimal 4
            4'b0101: reg_LEDs[6:2]=5'b00100; //7'b0010010 decimal 5
            4'b0110: reg_LEDs[6:2]=5'b00000; //7'b0000010 decimal 6
            4'b0111: reg_LEDs[6:2]=5'b11110; //7'b1111000 decimal 7
            4'b1000: reg_LEDs[6:2]=5'b00000; //7'b0000000 decimal 8
            4'b1001: reg_LEDs[6:2]=5'b00100; //7'b0010000 decimal 9
```

```
            4'b1010: reg_LEDs[6:2]=5'b10010; //7'b1001000 letter M
            4'b1011: reg_LEDs[6:2]=5'b00001; //7'b0000110 letter E
            4'b1100: reg_LEDs[6:2]=5'b10001; //7'b1000111 letter L
            4'b1101: reg_LEDs[6:2]=5'b10000; //7'b1000000 letter O
            4'b1110: reg_LEDs[6:2]=5'b01000; //7'b0100001 letter D
            4'b1111: reg_LEDs[6:2]=5'b11111; //7'b1111111 OFF
            default: reg_LEDs[6:2] = 5'b11111;

        endcase
    end
endmodule



module lab1part2(
    input [3:0] SW,
    output [6:0] HEX0
);

    seven_seg_decoder decoder(
        .x(SW),
        .hex_LEDs(HEX0)
    );

endmodule
```

Part 3: Seven-Segment Decoder and Counters

```
module lab1part3(
	input [9:0] SW,
	input [3:0] KEY,
	input CLOCK_50,
	output [6:0] HEX0
);

	// SW[9] and SW[8] for select line of mux4_1
	wire [1:0] x = {SW[9], SW[8]};

	// SW[3]-SW[0] for seven_seg_decoder
	wire [3:0] seg = {SW[3], SW[2], SW[1], SW[0]};

	// bits for 30-bit counter and MSB 4-bits
	wire [29:0] count30_out;
	wire [3:0] count30_msb = count30_out[29:26];

	// declare wire for 4-bit counter output
	wire [3:0] count4_out;

	// declare wire for mux output
	wire [3:0] mux_out;

	// instantiate mux
	mux4_1 mux(
		.select(x),
		.in_seg(seg),
		.in_count4(count4_out),
		.in_count30(count30_msb),
		.out(mux_out)
	);

	// instantiate 4-bit counter => counter # times KEY[3] pressed
	counter4 count4(
		.clk(KEY[3]),
		.reset_n(KEY[0]),
		.enable(1'b1),
		.out(count4_out)
	);

	// instantiate 30-bit counter
	counter30 count30(
```

```
                .clk(CLOCK_50),
                .reset_n(KEY[0]),
                .enable(1'b1),
                .result(count30_out)
    );

        // instantiate 7-seg
        seven_seg_decoder decoder(
                .x(mux_out),
                .hex_LEDs(HEX0)
        );

endmodule


// mux
module mux4_1(input [1:0] select, input [3:0] in_seg, input [3:0] in_count4, input [3:0]
in_count30, output reg [3:0] out);

        always@(*) begin
                case (select)
                        2'b00: out = in_seg; // 7-seg takes input from SW3-SW0
                        2'b01: out = in_count4; // 7-seg takes input from counter4
                        2'b10: out = in_count30; // takes MSB 4 input from counter30
                        2'b11: out = 4'b1111; // reset counters
                        default: out = 4'b1111; // default behaviour
                endcase
        end
endmodule



// 4-bit counter module with active low reset (bc keys are active low)
module counter4(input clk, input reset_n, input enable, output reg [3:0] out);
        always@(negedge clk or negedge reset_n) begin
                if (!reset_n) begin
                        out <= 4'b0000;
                end
                else if (enable) begin
                        out <= out + 1'b1;
                end
        end
endmodule
```

```
// 30-bit counter module
module counter30(input clk, input reset_n, input enable, output reg[WIDTH-1:0] result);
	parameter WIDTH = 30;
	//2’s power of 26 is 67,108,864.
	// that is 1 second (2^ 26/50e6 = 1.34), if count is using CLOCK 50
	always@(posedge clk, negedge reset_n)
		if (!reset_n) begin
			result <= 30'b0;
		end else if (enable) begin
			result <= result + 1'b1;
		end
endmodule


// 7-seg decoder module
module seven_seg_decoder(input [3:0] x, output [6:0] hex_LEDs);
   reg [6:0] reg_LEDs;

   assign hex_LEDs[0] = (~x[3] & ~x[2] & ~x[1] & x[0]) |
                        (x[2] & ~x[1] & ~x[0]) |
                        (x[3] & x[2] & x[1]);
   assign hex_LEDs[1] = (~x[3] & x[2] & ~x[1] & x[0]) |
                        (~x[3] & x[2] & x[1] & ~x[0]) |
                        (x[3] & x[2] & ~x[1] & ~x[0]) |
                        (x[3] & x[1] & x[0]);

   assign hex_LEDs[6:2]=reg_LEDs[6:2];

   always @(*)
   begin
      case (x)
         4'b0000: reg_LEDs[6:2]=5'b10000; //7'b1000000 decimal 0
         4'b0001: reg_LEDs[6:2]=5'b11110; //7'b1111001 decimal 1
         4'b0010: reg_LEDs[6:2]=5'b01001; //7'b0100100 decimal 2
         4'b0011: reg_LEDs[6:2]=5'b01100; //7'b0110000 decimal 3
         4'b0100: reg_LEDs[6:2]=5'b00110; //7'b0011001 decimal 4
         4'b0101: reg_LEDs[6:2]=5'b00100; //7'b0010010 decimal 5
         4'b0110: reg_LEDs[6:2]=5'b00000; //7'b0000010 decimal 6
         4'b0111: reg_LEDs[6:2]=5'b11110; //7'b1111000 decimal 7
         4'b1000: reg_LEDs[6:2]=5'b00000; //7'b0000000 decimal 8
         4'b1001: reg_LEDs[6:2]=5'b00100; //7'b0010000 decimal 9
         4'b1010: reg_LEDs[6:2]=5'b10010; //7'b1001000 letter M
```

```
            4'b1011: reg_LEDs[6:2]=5'b00001; //7'b0000110 letter E
            4'b1100: reg_LEDs[6:2]=5'b10001; //7'b1000111 letter L
            4'b1101: reg_LEDs[6:2]=5'b10000; //7'b1000000 letter O
            4'b1110: reg_LEDs[6:2]=5'b01000; //7'b0100001 letter D
            4'b1111: reg_LEDs[6:2]=5'b11111; //7'b1111111 OFF
            default: reg_LEDs[6:2] = 5'b11111;

        endcase
    end
endmodule
```